



Universidade do Porto

Faculdade de Engenharia

FEUP

Walking four legged Robot (Spider Mini)

Diogo Chaves, 202004540

Joana Pereira, 202004651

Relatório do Trabalho Prático no âmbito da unidade curricular Arquitetura de Computação Embarcada,
do 1ºano do Mestrado em Engenharia Eletrotécnica e de Computadores.

04/02/2024

Índice

1	Introdução	2
2	Componentes utilizados	3
3	Diagramas	4
3.1	Montagem dos componentes	4
3.2	Máquinas de estado	5
3.2.1	Máquina de estados principal	5
3.2.2	Funcionalidades extra	6
4	Implementação de software	6
4.1	Bibliotecas utilizadas	6
4.2	Moves	8
4.3	Implementação das máquinas de estado	9
4.3.1	Execução de movimentos	9
4.3.2	Correção da direção	10
4.3.3	Correção da inclinação	11
4.4	Main Loop	11
5	Conclusão	11

1 Introdução

O objetivo deste trabalho foi projetar um *robot* quadrúpede capaz de andar em linha reta contornando ou ultrapassando obstáculos e com a capacidade de subir escadas, utilizando um Raspberry Pi Pico W.

Começamos este projeto por controlar individualmente cada servo, para isso ligamos o componente ao *raspberry* de forma a percebermos o funcionamento do mesmo. Posteriormente montamos os servos na estrutura do *robot* como demonstrado na figura (1), para isso configuramos os servos a 90° e montamos as patas de modo a apontarem para as extremidades do *robot*. Após verificar o correto funcionamento, calibramos os servos de forma a mantê-los perfeitamente alinhados. Por último, juntamos o sensor de posição e o giroscópio.

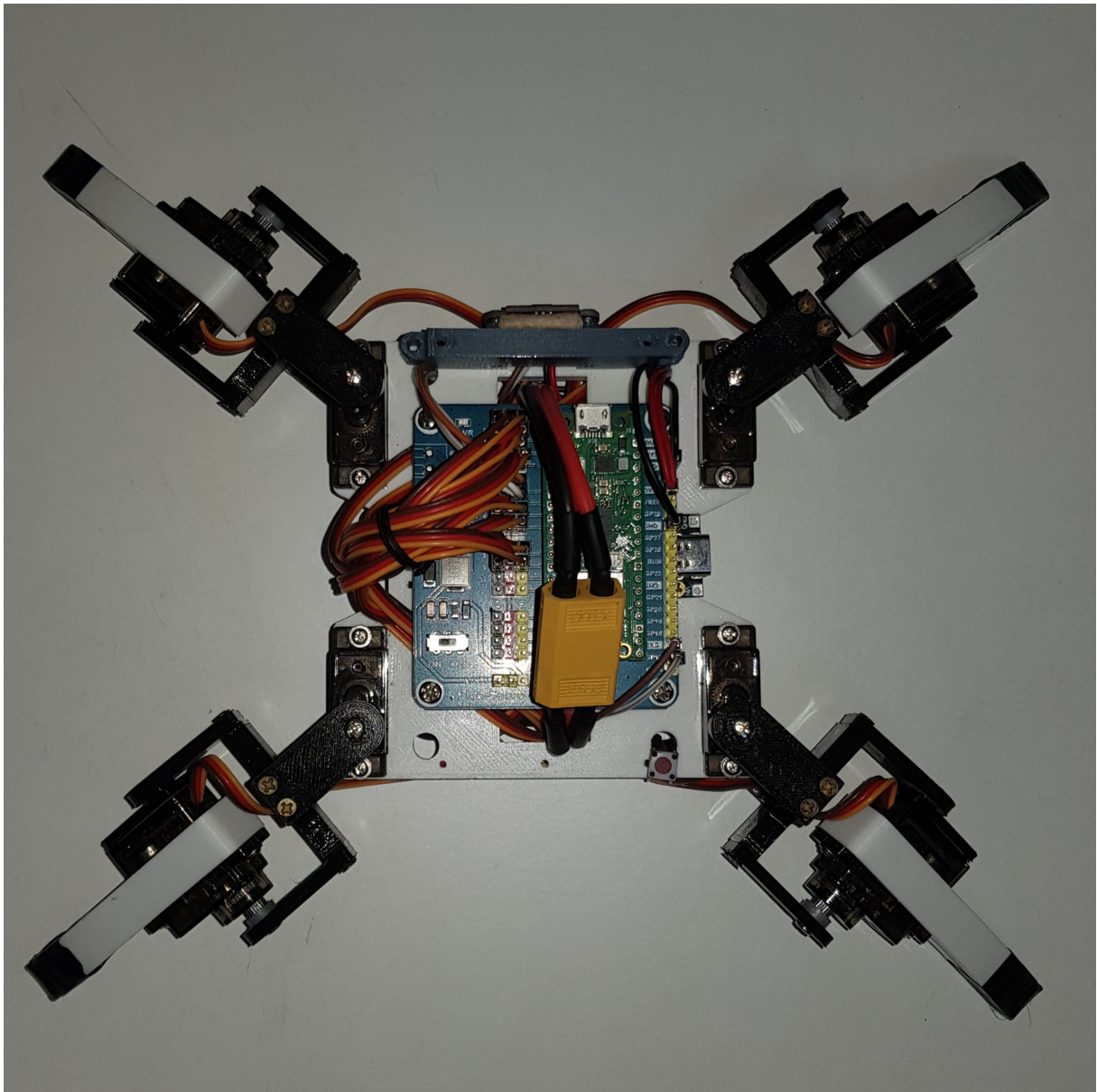


Figura 1: Spider-Mini

2 Componentes utilizados

- **VL53L0X**

O VL53L0X é um sensor de distância a laser (ToF) fabricado pela STMicroelectronics. Este sensor mede a distância entre si e um objeto, emitindo pulsos de luz infravermelha e medindo o tempo que a luz demora a viajar até ao objeto e voltar. Este sensor é conhecido pela precisão em medições de distância e é usado em diversas aplicações, como robótica e automação industrial. É compacto, seguro para a visão humana e comunica com outros dispositivos através de interfaces como I2C ou Serial.

- **MPU6050**

O MPU6050 é um sensor de movimento que combina um acelerômetro de três eixos e um giroscópio de três eixos num único chip. Este sensor é projetado para medir a aceleração e a taxa de rotação em diversos projetos eletrônicos, como robótica, drones, estabilizadores de câmera, entre outros.

Para utilizá-lo num projeto, é necessário conectar o MPU6050 a um micro-controlador ou a uma plataforma de desenvolvimento, como Arduino ou Raspberry Pi, e programar o código para ler e interpretar os dados fornecidos pelo sensor. Existem bibliotecas e exemplos de código disponíveis que facilitam na integração do MPU6050 em diferentes projetos.

- **BMS**

Uma BMS (Battery Management System) é um sistema essencial para o gestão de baterias recarregáveis, como as de lítio. As suas principais funções incluem monitorizar individualmente as células da bateria, equilibrando a carga para evitar disparidades de energia, proteção contra sobrecarga e descarga excessiva, controlo térmico para prevenir danos relacionados à temperatura, e gestão eficiente da corrente de carga e descarga. Neste projeto, utilizamos uma BMS bastante simples que apenas gere o carregamento das células da bateria de forma equilibrada.

- **Servo motores**

Um servo motor é um tipo de motor elétrico usado para controlar a posição de um objeto de maneira precisa e controlada. Este consiste num motor elétrico, uma caixa de engrenagens para redução de velocidade, um circuito de controlo interno com um micro-controlador e um sensor de posição. O sensor fornece *feedback* sobre a posição atual do eixo, permitindo ao micro-controlador comparar com a posição desejada. O controlo é geralmente realizado por meio de um sinal PWM, que modula a largura do pulso para determinar a posição. O servo motor é conhecido pela sua capacidade de manter posições com alta precisão, sendo amplamente utilizado em aplicações que requerem movimentos controlados, como robótica e automação industrial.

- **Raspberry Pi Pico**

O Raspberry Pi Pico é uma placa de desenvolvimento lançada pela Raspberry Pi Foundation. Baseado no micro-controlador RP2040, oferece 264 KB de RAM, pinos GPIO para conectividade, interfaces como I2C, SPI, UART, entradas/saídas analógicas e digitais, além de suporte para programação em MicroPython, C/C++ e Pico SDK. Com uma porta USB para alimentação e programação, é uma opção acessível e versátil para projetos eletrônicos e embarcados.

3 Diagramas

3.1 Montagem dos componentes

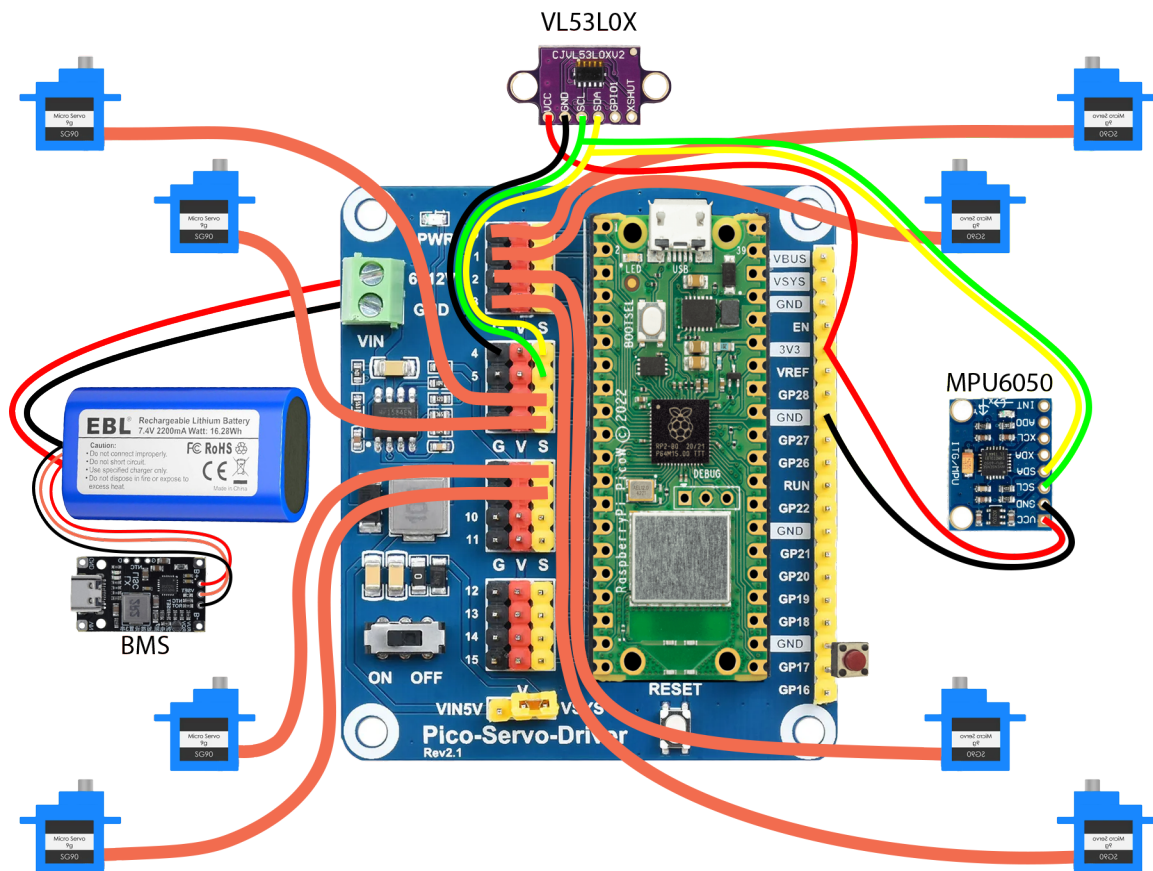


Figura 2: Representação da montagem do sistema

A figura 2 representa a montagem do sistema que constitui o *robot*. Os servos estão ligados nos pinos GPIO 0, 1, 2, 3, 6, 7, 8 e 9. Os pinos GP4 e GP5 ficaram reservados para a comunicação I2C com o sensor de obstáculos VL53L0X e com o giroscópio MPU6050. O botão para seleção do modo deve ligar o pino GP17 ao GND quando pressionado. O sensor e o giroscópio são ambos alimentados pelos 3.3V fornecidos pelo Raspberry Pi Pico, enquanto que os servos são alimentados pelos 5V provenientes da placa. Por fim, todo este sistema é alimentado por um *pack* de baterias de 2 células 18650 (gerando 7.4V) ao qual foi adicionado uma BMS com usb-C para carregamento de forma mais conveniente.

3.2 Máquinas de estado

3.2.1 Máquina de estados principal

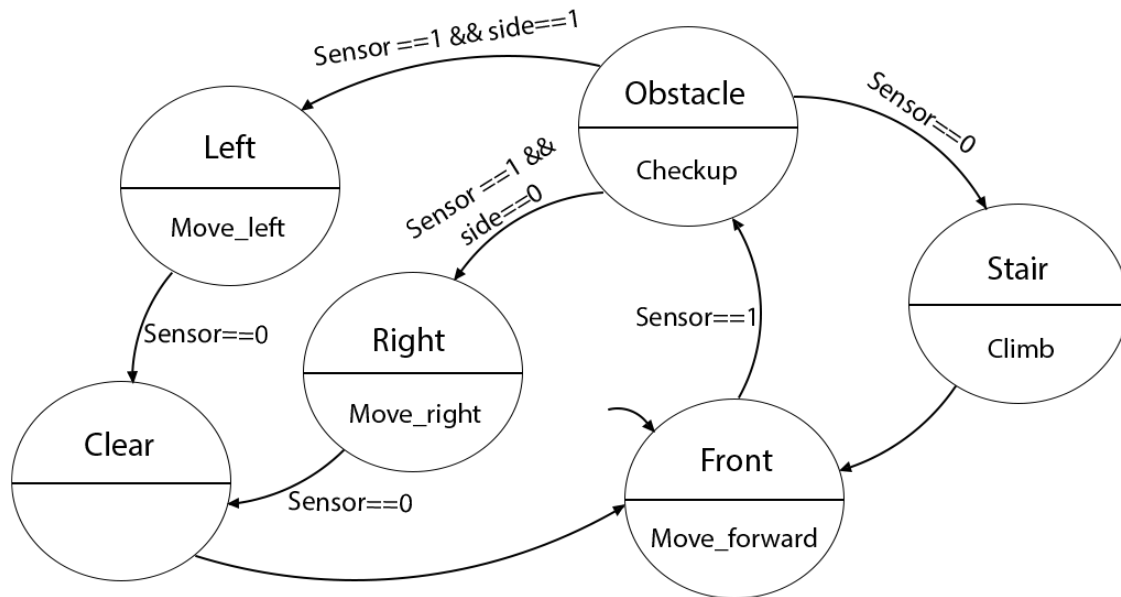


Figura 3: Máquina de estados principal

A figura 3 ilustra a máquina de estados implementada que controla a funcionalidade principal do *robot*, ou seja, andar em linha reta, desviar e ultrapassar obstáculos. O estado inicial do sistema é **Front**, onde o *robot* anda em frente. O sistema só transita para o estado **Obstacle** caso seja detetado algum obstáculo em frente ao *robot*. Neste estado, o sistema identifica o tipo de obstáculo. Caso o obstáculo seja baixo, como por exemplo uma pequena escada, o sistema transita para o estado **Stair**, sobe o obstáculo e transita novamente para o estado **Front**. No caso de o obstáculo ser alto, o sistema transita para o estado **Right** ou **Left** para se desviar andando para a direita ou esquerda, respetivamente. A primeira vez que ocorrer esta transição de estados, a transição será para o estado **Right**, a segunda transição será para o estado **Left** e assim vai alternando sucessivamente. Deste modo, garantimos que o *robot* se desvia dos obstáculos altos de forma alternada. A transição do estado **Right** ou **Left** para o estado **Clear** ocorre no momento em que o sensor frontal deixar de detetar qualquer objeto a obstruir a passagem. Neste estado, o *robot* certifica-se de que não vai bater com as patas no obstáculo dando mais 3 passos no sentido de se afastar e seguidamente a máquina de estados transita novamente para o estado **Front** que mantém o *robot* a andar na direção inicial em linha reta.

3.2.2 Funcionalidades extra

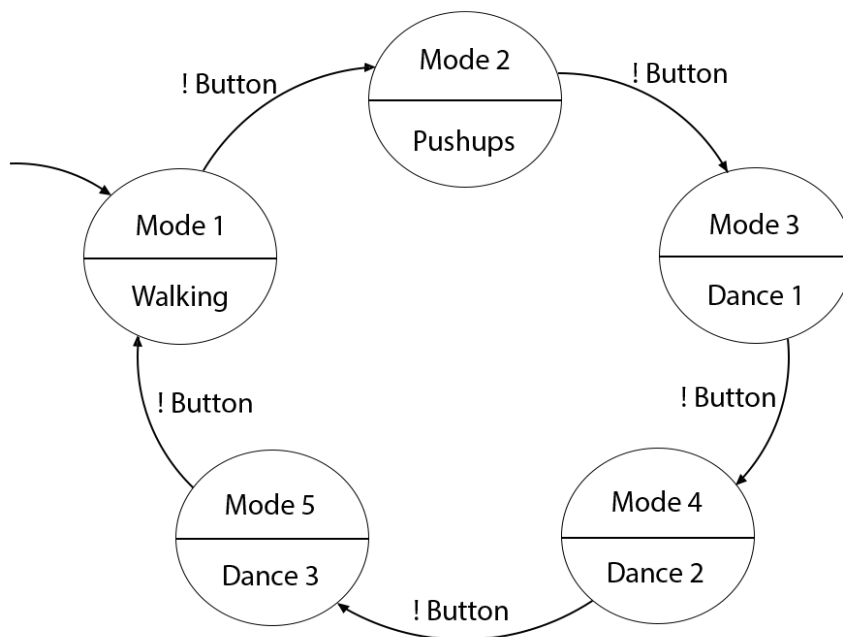


Figura 4: Máquina de estados adicional

Às funcionalidades pretendidas, acrescentamos outras habilidades ao *robot*. Para isso, adicionamos um botão para alternar entre os modos de funcionamento. A figura 4 representa a máquina de estados implementada para o *robot* executar o seu funcionamento principal (Mode 1), referido na secção 3.2.1, flexões (Mode 2) e 3 danças diferentes que podem ser observadas alternando entre os restantes modos. A transição entre estados / modos de funcionamento ocorre sempre que é pressionado o botão adicional que se encontra na zona anterior do *robot*.

4 Implementação de software

4.1 Bibliotecas utilizadas

Para a implementação do software, utilizamos as seguintes bibliotecas:

- **Servo**

A biblioteca Servo.h é utilizada para controlar os servos. Um servo é ativado criando uma instância da classe Servo e passando o pino GPIO ao método:

```
void Servo::attach(int pin);
```

A posição destes motores varia entre 0 e 180°. Esta biblioteca permite gerar um sinal PWM que roda os servos para a posição desejada, em graus, através do método:

```
void Servo::write(int value);
```

- **VL53L0X**

Esta biblioteca facilita a integração e uso do sensor VL53L0X. Para inicializar o sensor de distância criamos uma instância da classe VL53L0X e utilizamos o seguinte método para inicializar o sensor:

```
bool VL53L0X::init(bool io_2v8 = true);
```

No argumento `io_2v8` deve ser introduzido o endereço do dispositivo a conectar. Este método retorna 0 ou 1 no caso da inicialização não ser ou ser bem sucedida, respetivamente.

Para detetar a distância utilizamos o seguinte método, que realiza uma medição única e retorna a leitura em milímetros:

```
uint16_t VL53L0X::readRangeSingleMillimeters()
```

Com esta biblioteca conseguimos controlar o *robot*, de modo a detetar os obstáculos a uma distância definida para os contornar ou ultrapassar.

• MPU6050_6Axis_MotionApps20

A biblioteca MPU6050_6Axis_MotionApps20 é projetada para facilitar a comunicação com o MPU6050, e fornece funcionalidades avançadas para a deteção de movimento, nomeadamente quando o *robot* está inclinado. Esta biblioteca é bastante intuitiva, criamos uma instância da classe MPU6050 e utilizamos os seguintes métodos para inicializar e testar o sensor:

```
void MPU6050::initialize();  
bool MPU6050::testConnection();
```

Há duas formas de extrair valores do MPU6050. Uma forma é ler os valores *raw* do sensor e usar os valores para calcular a nova orientação. Outra forma e também a escolhida para este projeto, é retirar os valores diretamente do processador DMP (Digital Motion Processor).

Para inicializar o DMP no giroscópio, utilizamos o seguinte método:

```
uint8_t MPU6050::dmpInitialize();
```

Os seguintes métodos foram utilizados para realizar a calibração automática do acelerómetro e do giroscópio.

```
void MPU6050::CalibrateGyro(uint8_t Loops = 15);  
void MPU6050::CalibrateAccel(uint8_t Loops = 15);
```

Também é possível definir manualmente os offsets em alternativa à calibração automática através dos seguintes métodos:

```
void MPU6050::setXAccelOffset(int16_t offset);  
void MPU6050::setYAccelOffset(int16_t offset);  
void MPU6050::setZAccelOffset(int16_t offset);  
void MPU6050::setXGyroOffset(int16_t offset);  
void MPU6050::setYGyroOffset(int16_t offset);  
void MPU6050::setZGyroOffset(int16_t offset);
```

Por fim, ativamos o DMP através do seguinte método para posteriormente ser possível ler os valores nele contidos:

```
void MPU6050::setDMPEntered(bool entered);
```

Quanto às leituras das inclinações e rotações em relação aos eixos dos X, Y e Z, optamos pelo método baseado nos ângulos de Euler. Para obter os valores desejados, primeiro é necessário ler os últimos valores presentes no DMP através da seguinte função:

```
void MPU6050::dmpGetCurrentFIFOpacket(uint8_t *data);
```

Os seguintes métodos utilizam os valores presentes no pacote lido do DMP para calcular a orientação do sensor em relação aos seus eixos:

```
uint8_t MPU6050::dmpGetQuaternion(Quaternion *q, const uint8_t* packet=0);
uint8_t MPU6050::dmpGetGravity(VectorFloat *v, Quaternion *q);
uint8_t MPU6050::dmpGetYawPitchRoll(float *data, Quaternion *q, VectorFloat *gravity);
```

Neste caso, os ângulos ficariam guardados no vetor *data* com os valores em radianos. Ou seja, *data*[0] retorna a rotação em torno de Z, *data*[1] a rotação em torno do eixo Y e *data*[2] em torno de X.

• PID_v1

Esta biblioteca fornece uma implementação para o controlo PID, facilitando a configuração e uso deste método de controlo. Neste projeto, usamos PID para ajustar a trajetória do *robot*, de forma a andar em linha reta compensando os desvios que vai tendo ao longo do seu percurso. Para além disso, também usamos um PID para manter o corpo do robot paralelo ao chão, quando se encontra num plano inclinado ou a subir algum obstáculo.

Para instanciar um PID temos de recorrer ao seguinte construtor, onde definimos a entrada e a saída do sistema, o valor desejado para a orientação (*setpoint*), os ganhos proporcional, integral e derivativo e o modo de operação, que pode ser proporcional direto ou inverso. Neste projeto, utilizamos o modo de operação direto, que ajusta a saída do controlador quando a entrada está abaixo ou acima do *setpoint*.

```
PID::PID( double* inputVar, double* outputVar, double* setpointVar, double
         kp, double ki, double kd, int P0n);
```

Posto isto, utilizamos o seguinte método para definir o modo de operação do PID, que pode ser automático, onde a saída é ajustada automaticamente pelo controlador PID para atingir o *setpoint*, ou manual, onde a saída é definida manualmente. Para este projeto nós utilizamos o modo de operação automático.

```
void PID::SetMode(int Mode);
```

A seguinte função é usada para definir limites para a saída do controlador PID. Esta é bastante útil para garantir que a saída gerada pelo controlador não ultrapasse a gama de valores desejada.

```
void PID::SetOutputLimits(double outMin, double outMax);
```

Para calcular a saída do controlador PID com base no erro atual, utilizamos o seguinte método. Esta função tem de ser chamada num *loop* para que os valores da saída sejam atualizados continuamente.

```
void PID::bool Compute();
```

• Wire

A biblioteca *wire.h* é normalmente utilizada para a comunicação I2C, neste caso entre o *Raspberry* e os sensores. Para inicializar o barramento I2C, utilizamos o seguinte método:

```
void Wire::begin()
```

4.2 Moves

Para definir os movimentos que o *robot* irá realizar, criamos uma biblioteca (*moves.h*). Nesta biblioteca encontram-se estruturados os *arrays* bidimensionais que dão origem aos movimentos do *robot*, como, andar em frente, andar para a esquerda, dançar, etc. Cada *array* representa um conjunto de movimentos para controlar cada servo. As 8 primeiras colunas desta matriz, representam as posições angulares de cada servo, e a última representa a duração do passo em milissegundos. Cada linha contém um passo do movimento, sendo que a primeira linha corresponde à posição inicial de cada servo e as seguintes representam uma soma ou subtração ao ângulo definido inicialmente, de forma ao *robot* descrever o movimento corretamente.

```

int Forward [][][numberOfACE] = {
    // GP0, GP1, GP2, GP3, GP6, GP7, GP8, GP9, ms
    { 30, 90, 90, 150, 150, 90, 90, 30, 100 },
    { 30, 0, 0, 0, 0, 0, -55, 30, 100 },
    { -30, 0, 0, 0, 0, 0, 0, -30, 100 },
    { 0, 0, 0, -30, -30, 0, 0, 0, 100 },
    { 0, -55, 55, 0, 0, 0, 55, 0, 100 },
    { 0, 0, 0, 30, 30, 0, 0, 0, 100 },
    { 30, 55, 0, 0, 0, 0, 0, 30, 100 },
    { 0, 0, -55, 0, 0, 55, 0, 0, 100 },
    { -30, 0, 0, 0, 0, 0, 0, -30, 100 },
    { 0, 0, 0, 0, -30, 0, 0, 0, 100 },
    { 0, 0, 0, 0, 30, -55, 0, 0, 100 },
};

```

Este é o *array* com as posições angulares para o *robot* executar o movimento de andar em frente. Ou seja, após o *array* ter sido todo percorrido o *robot* terá dado um passo para a frente.

4.3 Implementação das máquinas de estado

As máquinas de estado descritas na secção 3.2 do projeto, estão implementadas no ficheiro 'main.cpp' em duas funções, nomeadamente void SpiderMini() e void ModeSelect(). Apenas esta última função é chamada no *main loop* do programa e é responsável por alterar o modo de configuração do *robot*, sendo que o primeiro estado desta máquina executa a máquina de estados principal, ou seja, a função SpiderMini(). Os estados das duas máquinas estão definidos nos seguintes *enums*:

```

typedef enum{ // Main state machine
    Front,
    Right,
    Left,
    Clear,
    Obstacle,
    Stair
} state;

typedef enum{ // Mode select state machine
    Mode1,
    Mode2,
    Mode3,
    Mode4,
    Mode5,
} operation_mode;

```

Desta forma, simplificamos o acompanhamento do estado atual das máquinas de estado e melhoramos a legibilidade do código.

4.3.1 Execução de movimentos

A seguinte função é bastante importante para o *robot* realizar qualquer movimento, pois é através dela que escolhemos o movimento a executar. Para isso, temos de lhe passar dois parâmetros. Um deles é o *array* bidimensional correspondente ao movimento que queremos realizar, e o outro é o número de passos que este movimento tem, ou seja o número de linhas que o *array* selecionado tem.

```
void runServoPrgV(int servoPrg[][numberOfACE], int step)
```

Dentro desta função, primeiro começamos por obter a posição inicial dos servo. Com um ciclo *for* percorremos cada servo, onde lemos a sua posição atual e armazenamos o seu valor subtraído ao valor definido para a calibração num vetor auxiliar.

```
for (int s = 0; s < numberOfServos; s++)
    servoPos[s] = servo[s].read() - servoCal[s];
```

Posto isto calculamos a posição final dos servos. Num ciclo *for* percorremos novamente os servos, onde calculamos, através do *array* fornecido à função, a posição a que cada servo se deve encontrar no final de cada passo.

Por último, interpolamos a posição angular de cada servo, entre a posição inicial e final, e por fim atualizamos a posição dos servos, ou seja escrevemos em cada servo o ângulo desejado tendo em conta o tempo definido para esse passo do movimento.

```
servo[k].write((map(j, 0, totalTime / servoPrgPeriod, servoPos[k],
    servoPrevPrg[k]) + servoCal[k]));
```

4.3.2 Correção da direção

Uma das funcionalidades do *robot* era andar numa linha reta. Utilizamos então os valores da orientação do giroscópio para atualizar os valores do *array*, de forma a compensar as variações resultantes das folgas existentes, tanto nos servos como nas estruturas que os suportam e também derivadas à falta de aderência em certos pavimentos. A forma mais eficaz de executar estas correções foi ler o ângulo da direção devolvido pelo osciloscópio, utilizar um PID para calcular a compensação necessária nos movimentos (Output) e por fim criamos uma função adicional para somar/subtrair esse mesmo Output aos valores presentes no *array*. Abaixo está o exemplo de como a atualização é executada para o caso de andar para a frente:

```
void MoveFrontUpdate(){
    Forward[4][1] = -45 + Output;
    Forward[6][1] = 45 - Output;
    Forward[4][2] = 45 - Output;
    Forward[7][2] = -45 + Output;
    Forward[7][5] = 45 + Output;
    Forward[10][5] = -45 - Output;
    Forward[1][6] = -45 - Output;
    Forward[4][6] = 45 + Output;
}
```

Desta forma conseguimos manter o *robot* a andar sempre numa linha reta, tanto em frente como para os lados quando se está a desviar de um obstáculo alto. É de notar que a direção em que o *robot* se irá deslocar é a direção para a qual o *robot* está virado no momento em que é iniciado.

4.3.3 Correção da inclinação

Quanto à correção da inclinação do *robot* quando se encontra em planos inclinados ou a subir degraus, a execução é semelhante à referida na secção 4.3.2, mas neste caso é calculado o valor da inclinação do *robot* com o auxílio do giroscópio.

A maneira mais eficaz de executar esta correção foi alterar apenas as colunas da primeira linha do *array* responsáveis pelo controlo dos servos que fazem subir e descer as patas:

```
void ClimbUpdate(){
    Forward[0][0] = 30 + Climb_Output;
    Forward[0][3] = 150 + Climb_Output;
    Forward[0][4] = 150 - Climb_Output;
    Forward[0][7] = 30 - Climb_Output;
}
```

A variável *Climb_Output* é a correção calculada pelo PID responsável pelas mudanças de inclinação.

4.4 Main Loop

Após o código estar encapsulado tanto em funções como nas respetivas máquinas de estado, a função *loop* fica bastante simples e compacta. Nesta secção do código é chamada a função para o cálculo dos valores do giroscópio, a função para atualizar a saída dos PIDs e a máquina de estados que executa os modos.

Caso não sejam necessárias funcionalidades adicionais, a função *ModeSelect()* pode ser substituída pela função *Spider-Mini()* e deste modo o *robot* fica apenas com o modo de funcionamento principal.

```
mpuGetValues(); //get values from mpu
myPID.Compute(); //compute PID
myPID2.Compute(); //compute PID2
ModeSelect(); //mode select
```

5 Conclusão

O desenvolvimento deste projeto revelou-se bastante desafiador e empolgante, pois através dele conseguimos interligar vários conhecimentos adquiridos nesta e noutras unidades curriculares, melhorando a nossa capacidade de programar e desenvolver sistemas embarcados. Através deste trabalho, descobrimos um grande interesse no desenvolvimento de projetos na área de robótica. Foi uma excelente oportunidade para descobrir as bibliotecas disponíveis no ecossistema do Arduino e também aprender de forma prática como funciona a interface I2C no Raspberry Pi Pico.

Para concluir, este projeto foi desenvolvido de forma consistente, cumprindo todas as funcionalidades propostas e não apresenta qualquer *bug*.